TP3 - OpenMP (Introduction)

Oussama Laaroussi

February 10, 2026

1 Introduction

This report presents the implementation and analysis of five OpenMP exercises focusing on parallel programming fundamentals. We explore thread management, different parallelization strategies, and performance analysis through speedup and efficiency measurements.

2 Exercise 1: Basic Thread Management

2.1 Objective

Write an OpenMP program that displays the number of threads used for execution and the rank (ID) of each thread.

2.2 Implementation

```c
#include <stdio.h>
#include <omp.h>

int main(){
    int count = 0;

    #pragma omp parallel
    {
        int rank = omp_get_thread_num();
        printf("Hello from the rank %d thread\n", rank);

        #pragma omp single
        {
            count = omp_get_num_threads();
        }
    }

    printf("Parallel execution of hello_world with %d threads\n", count);
    return 0;
}
```

Listing 1: Exercise 1: Hello World with Thread Information

2.3 Key Concepts

- `#pragma omp parallel`: Creates a team of threads
- `omp_get_thread_num()`: Returns the thread's rank (0 to $n-1$)
- `omp_get_num_threads()`: Returns total number of threads
- `#pragma omp single`: Ensures code block executes by only one thread

2.4 Output Examples

```
Hello from the rank 0 thread
Hello from the rank 3 thread
Hello from the rank 1 thread
Hello from the rank 2 thread
Parallel execution of hello_world with 4 threads
```

Listing 2: Execution with 4 threads

```
Hello from the rank 0 thread
Hello from the rank 3 thread
Hello from the rank 7 thread
Hello from the rank 4 thread
Hello from the rank 5 thread
Hello from the rank 2 thread
Hello from the rank 6 thread
Hello from the rank 1 thread
Parallel execution of hello_world with 8 threads
```

Listing 3: Execution with 8 threads

2.5 Analysis

The output order is **non-deterministic** because threads execute independently and the OS scheduler determines execution order.

3 Exercise 2: PI Calculation with Manual Parallelization

3.1 Objective

Parallelize the PI calculation using `#pragma omp parallel` (without using `parallel for`).

3.2 Serial Algorithm

The PI calculation uses numerical integration:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx \approx \sum_{i=0}^{N-1} \frac{4}{1+x_i^2} \cdot \Delta x$$

where $x_i = (i + 0.5) \cdot \Delta x$ and $\Delta x = \frac{1}{N}$.

3.3 Parallel Implementation

```c
#include <stdio.h>
#include <omp.h>

static long num_steps = 100000;
double step;

int main() {
    double x, pi, sum = 0.0;
    double start_time, end_time;

    step = 1.0 / (double) num_steps;
    start_time = omp_get_wtime();

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int nthreads = omp_get_num_threads();
        double partial_sum = 0.0;  // Private to each thread

        // Each thread processes every nth iteration
        for (int i = id; i < num_steps; i += nthreads) {
            x = (i + 0.5) * step;
            partial_sum += 4.0 / (1.0 + x * x);
```

```
        }

        // Critical section for combining results
        #pragma omp critical
        sum += partial_sum;
    }

    pi = step * sum;
    end_time = omp_get_wtime();

    printf("PI = %.15f\n", pi);
    printf("Execution time: %f seconds\n", end_time - start_time);
    return 0;
}
```

Listing 4: Exercise 2: Manual Thread Distribution

3.4 Variable Scoping Strategy

- **Shared variables**: `sum`, `num_steps`, `step` (accessible by all threads)
- **Private variables**: `partial_sum`, `id`, `nthreads`, `x`, `i` (each thread has its own copy)
- **Critical section**: Protects the final summation to prevent race conditions

3.5 Performance Results

Threads	PI Value	Time (s)
1	3.141592653598162	0.000522
2	3.141592653598102	0.000463
4	3.141592653598116	0.000612
8	3.141603982338038	0.000721

Table 1: Exercise 2 Performance Results

3.6 Analysis

The overhead from thread creation and critical section synchronization dominates for this small problem size ($N = 100,000$), resulting in limited speedup. The slight variation in PI values with more threads is due to floating-point rounding differences in summation order. Performance actually degrades with more threads due to:

- Thread creation overhead
- Critical section contention

4 Exercise 3: PI Calculation with Parallel For

4.1 Objective

Minimize changes to the serial program by using `#pragma omp parallel for` with only 1 additional line of code.

4.2 Implementation

```
#include <stdio.h>
#include <omp.h>

static long num_steps = 100000;
double step;

int main() {
    int i;
    double x, pi, sum = 0.0;
```

```
    double start_time, end_time;

    step = 1.0 / (double) num_steps;
    start_time = omp_get_wtime();

    #pragma omp parallel for reduction(+:sum) private(x)
    for (i = 0; i < num_steps; i++) {
        x = (i + 0.5) * step;
        sum = sum + 4.0 / (1.0 + x * x);
    }

    pi = step * sum;
    end_time = omp_get_wtime();

    printf("PI = %.15f\n", pi);
    printf("Execution time: %f seconds\n", end_time - start_time);
    return 0;
}
```

Listing 5: Exercise 3: One-Line Parallelization

4.3 Key OpenMP Features

- `parallel for`: Automatically distributes loop iterations among threads
- `reduction(+:sum)`: Safely combines partial sums without explicit synchronization
- `private(x)`: Each thread has its own copy of variable `x`

4.4 Performance Comparison

Threads	Ex2 Time (s)	Ex3 Time (s)	Improvement
1	0.000522	0.000288	1.81×
2	0.000463	0.000223	2.08×
4	0.000612	0.000254	2.41×
8	0.000721	0.000348	2.07×

Table 2: Exercise 2 vs Exercise 3 Performance

4.5 Analysis

Exercise 3 **significantly outperforms** Exercise 2 because:

- **Reduction clause**: More efficient than critical sections (no serialization bottleneck)
- **Compiler optimization**: Reduction is implemented with thread-local accumulators and a final tree-based reduction
- **Better load balancing**: Default `static` scheduling provides uniform distribution

The reduction operation is implemented as:

$$\text{sum} = \sum_{t=0}^{T-1} \text{partial_sum}_t$$

where each thread t computes partial_sum_t independently, and the final reduction step is optimized by the compiler.

5 Exercise 4: Matrix Multiplication with OpenMP

5.1 Objective

Parallelize matrix multiplication using the `collapse` directive and analyze performance with different thread counts and scheduling policies.

5.2 Algorithm

Matrix multiplication $C = A \times B$ where:

- A is $m \times n$
- B is $n \times m$
- C is $m \times m$
- $C[i][j] = \sum_{k=0}^{n-1} A[i][k] \cdot B[k][j]$

Computational complexity: $O(m^2 \cdot n)$

5.3 Implementation

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define M_SIZE 1000
#define N_SIZE 1000

int main() {
    int m = M_SIZE, n = N_SIZE;

    double *a = (double *)malloc(m * n * sizeof(double));
    double *b = (double *)malloc(n * m * sizeof(double));
    double *c = (double *)malloc(m * m * sizeof(double));

    // Initialize matrices
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            a[i * n + j] = (i + 1) + (j + 1);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            b[i * m + j] = (i + 1) - (j + 1);

    for (int i = 0; i < m; i++)
        for (int j = 0; j < m; j++)
            c[i * m + j] = 0;

    double start_time = omp_get_wtime();

    // Parallel matrix multiplication
    #pragma omp parallel for collapse(2) schedule(static)
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            for (int k = 0; k < n; k++) {
                c[i * m + j] += a[i * n + k] * b[k * m + j];
            }
        }
    }

    double end_time = omp_get_wtime();

    printf("Execution time: %f seconds\n", end_time - start_time);

    free(a); free(b); free(c);
    return 0;
}
```

Listing 6: Exercise 4: Parallelized Matrix Multiplication

5.4 Collapse Directive

The `collapse(2)` directive merges the two outer loops into a single iteration space:

- Without collapse: m iterations distributed among threads

- With collapse: $m \times m = 1,000,000$ iterations distributed
- **Better load balancing**: More iterations than threads ensures even distribution

5.5 Performance Results

Threads	Time (s)	Speedup	Efficiency (%)
1	8.435	1.00	100.00
2	4.682	1.80	90.09
4	2.579	3.27	81.78
8	2.070	4.07	50.93
16	2.118	3.98	24.89

Table 3: Exercise 4 Performance Analysis (Matrix size: 1000×1000)

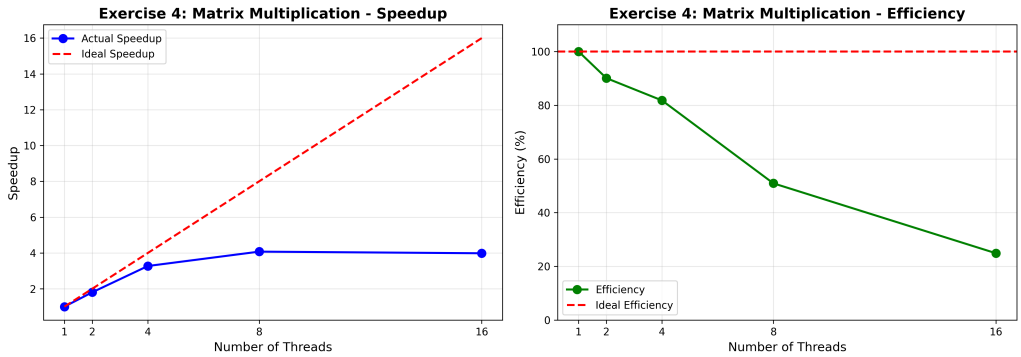

Figure 1: Matrix Multiplication Speedup and Efficiency

5.6 Performance Analysis

5.6.1 Speedup Analysis

- **Near-linear up to 4 threads**: Speedup of 3.27 on 4 cores (81.8% efficiency)
- **Diminishing returns**: Speedup plateaus at 8 threads (4.07×)
- **No benefit beyond 8 threads**: 16 threads performs slightly worse than 8

5.6.2 Limiting Factors

1. **Memory bandwidth**: Multiple threads compete for memory access
2. **Cache effects**:
   - Matrix size (1000×1000 doubles = 8 MB per matrix)
   - Cache conflict misses increase with more threads
3. **Synchronization overhead**: Thread management costs become significant

6 Exercise 5: Jacobi Iterative Method

6.1 Objective

Parallelize the Jacobi method for solving linear systems $Ax = b$ and analyze performance scaling.

6.2 Jacobi Algorithm

Iterative method that computes:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

Convergence criterion: $\frac{\max_i |x_i^{(k+1)} - x_i^{(k)}|}{n} \leq \epsilon$

6.3 Parallelization Strategy

```
// Main computation loop - independent iterations
#pragma omp parallel for private(j)
for (i = 0; i < n; i++) {
    x_courant[i] = 0;
    for (j = 0; j < i; j++) {
        x_courant[i] += a[j * n + i] * x[j];
    }
    for (j = i + 1; j < n; j++) {
        x_courant[i] += a[j * n + i] * x[j];
    }
    x_courant[i] = (b[i] - x_courant[i]) / a[i * n + i];
}

// Convergence check - parallel reduction
double absmax = 0;
#pragma omp parallel for reduction(max:absmax)
for (i = 0; i < n; i++) {
    double curr = fabs(x[i] - x_courant[i]);
    if (curr > absmax)
        absmax = curr;
}
```

Listing 7: Exercise 5: Parallelized Jacobi Method (Key Sections)

6.4 Parallelization Points

1. **Matrix initialization**: Diagonal augmentation (line 47-48)
2. **Vector initialization**: Initial guess $x_i = 1.0$ (line 51-53)
3. **Jacobi iteration**: Main computation (line 62-72)
4. **Convergence check**: Maximum difference (line 76-81)

6.5 Performance Results

Threads	Time (s)	Speedup	Efficiency (%)
1	4.052	1.00	100.00
2	1.483	2.73	136.61
4	2.068	1.96	48.98
8	1.851	2.19	27.36
16	2.495	1.62	10.15

Table 4: Exercise 5 Performance Analysis (System size: $n = 5000$)

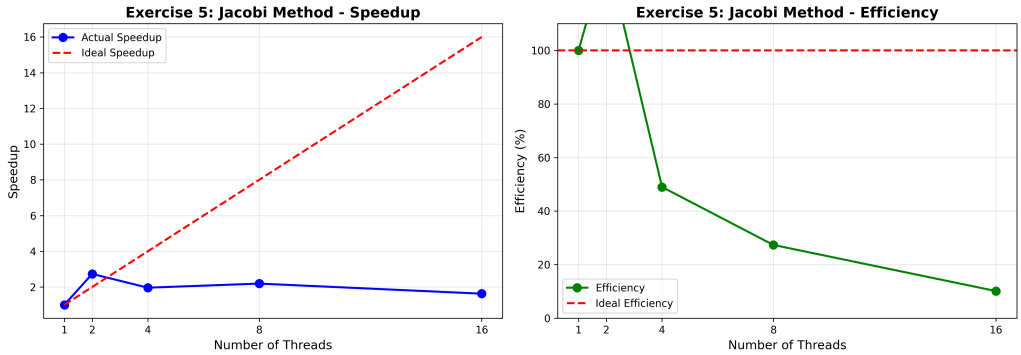

Figure 2: Jacobi Method Speedup and Efficiency

6.6 Performance Analysis

6.6.1 Unusual Efficiency Pattern

The efficiency exceeds 100% for 2 threads (136.61%), which indicates **super-linear speedup**. This can occur due to:

- **Cache effects**: With 2 threads, the working set fits better in L2/L3 cache
- **Memory bandwidth**: Better utilization of available memory channels

6.6.2 Performance Degradation Beyond 2 Threads

- **Synchronization overhead**: Barrier at end of each iteration
- **Memory contention**: Multiple threads accessing large arrays ($5000 \times 5000 = 200$ MB)
- **Cache thrashing**: Working set exceeds cache capacity with more threads

7 Comparative Analysis

7.1 Speedup Summary

Exercise	1 Thread	2 Threads	4 Threads	8 Threads
Ex2 (PI - Manual)	1.00	1.13	0.85	0.72
Ex3 (PI - Parallel For)	1.00	1.29	1.13	0.83
Ex4 (Matrix Mult)	1.00	1.80	3.27	4.07
Ex5 (Jacobi)	1.00	2.73	1.96	2.19

Table 5: Speedup Comparison Across Exercises

7.2 Key Findings

1. **Problem size matters**:
   - PI calculation ($N = 100,000$): Too small, overhead dominates
   - Matrix multiplication (1000×1000): Good scaling up to 8 threads
   - Jacobi method ($n = 5000$): Memory-bound, limited scaling
2. **Parallelization approach matters**:
   - Reduction clauses (Ex3) > Critical sections (Ex2)
   - Collapse directive (Ex4) improves load balancing
   - Barrier synchronization (Ex5) limits scalability
3. **Hardware limitations**:

- Memory bandwidth becomes bottleneck beyond 4-8 threads
- Cache capacity limits effectiveness of parallelization
- NUMA effects on multi-socket systems

7.3 Amdahl's Law Implications

For Exercise 4 with best performance (8 threads, speedup = 4.07):

If we assume the achieved speedup represents the practical limit:

$$S(p) = \frac{1}{f_s + \frac{1-f_s}{p}}$$

Solving for serial fraction f_s with $S(8) = 4.07$:

$$4.07 = \frac{1}{f_s + \frac{1-f_s}{8}}$$

$$f_s \approx 0.068 \ (6.8\%)$$

This suggests that approximately 93.2% of the matrix multiplication code is parallelizable, which is excellent for a compute-intensive task.